

DAY 6 ... TASK 6:

BUILDING PYTHON LIKE A PUZZLE – COMPOSITION, AGGREGATION, & CLASS POWER

When Objects Use Other Objects Mastering Composition, Aggregation & Class Mechanics in Python OOP

✓ Assigned Task 6

❖ Theoretical Assignment

1. Composition vs Aggregation

- Define composition with a real-life example.
 - Define aggregation with a real-life example.
 - Difference between composition and aggregation in Python.
 - Why and when to use each?
 - Explain with a UML-style structure using plain text or drawing.
-

2. Class vs Static Variables

- Define class variable and static variable.
 - What is the difference between instance variable and class variable?
 - How do you declare static variables?
 - What is the scope and behavior of each?
-

3. Method Resolution Order (MRO)

- What is MRO in Python?
 - How does MRO work in multiple inheritance?
 - Explain the **C3 Linearization** algorithm (used in Python MRO).
 - How do you use Class.__mro__ and super()?
-

❖ Practical Coding Task

Build a Company-Employee System Demonstrating Composition & Aggregation

Step-by-step:

1. Composition:

- Create a Department class.
- Create an Employee class.
- Each Employee should have a Department object inside it (composition).
- If Employee is deleted, Department remains.

2. Aggregation:

- Create a Project class.
- Allow multiple employees to work on a project (store them in a list).
- Even if a project is deleted, employee objects should remain active.

3. Class vs Static Variables:

- Use a class variable to count how many employees are created.
- Use a static method to display the company slogan.

4. MRO & super():

- Create a class structure like:
- class A: ...
- class B(A): ...
- class C(A): ...
- class D(B, C): ...
- Use super() in constructors and methods.
- Print the MRO of class D.

➔ Bonus Tip

“When you understand how objects work together (composition/aggregation), you’re not just coding you’re architecting!”

 Deadline:

*Submit before: **Tomorrow night by 12:59 AM** (before Day 7 starts)*

Final:

-> Once you finish this task, send the message:

“ **Task 6 Completed**”

Your journey to full OOP mastery is almost complete.... stay focused and powerful!